Project Report: Automated Data Extraction from Partial Tax Invoices using YOLO

1. Introduction & Objectives

The goal of this project is to build an object detection model capable of reading and extracting specific line-item data from Thai partial tax invoices (receipts). Rather than simply running raw Optical Character Recognition (OCR) on the whole document, I am training a YOLO-based model to locate and classify four specific bounding boxes first:

- `quantity_item`: The number of items purchased
- `product_description`: The name/details of the item (primarily in Thai)
- `price`: The total price for that specific line item
- `total_due_amount`: The final grand total of the receipt

By isolating these regions first, the data pipeline becomes much more structured and accurate.

2. Data Collection

To ensure the model generalizes well, I collected a diverse dataset of roughly 100 receipts from my own personal purchases.

- **Physical Receipts:** The majority of the dataset consists of paper receipts from convenience stores and supermarkets like 7-Eleven and Tops (e.g., Tops Belle Grand Rama 9). Because paper receipts get crumpled or have varying lighting, I used the CamScanner app to digitize them. This ensured the text was relatively flat but still retained real-world artifacts.
- **Digital Receipts:** I also included digital e-receipts (like from Uniqlo) to introduce clean, perfectly aligned text into the training set.

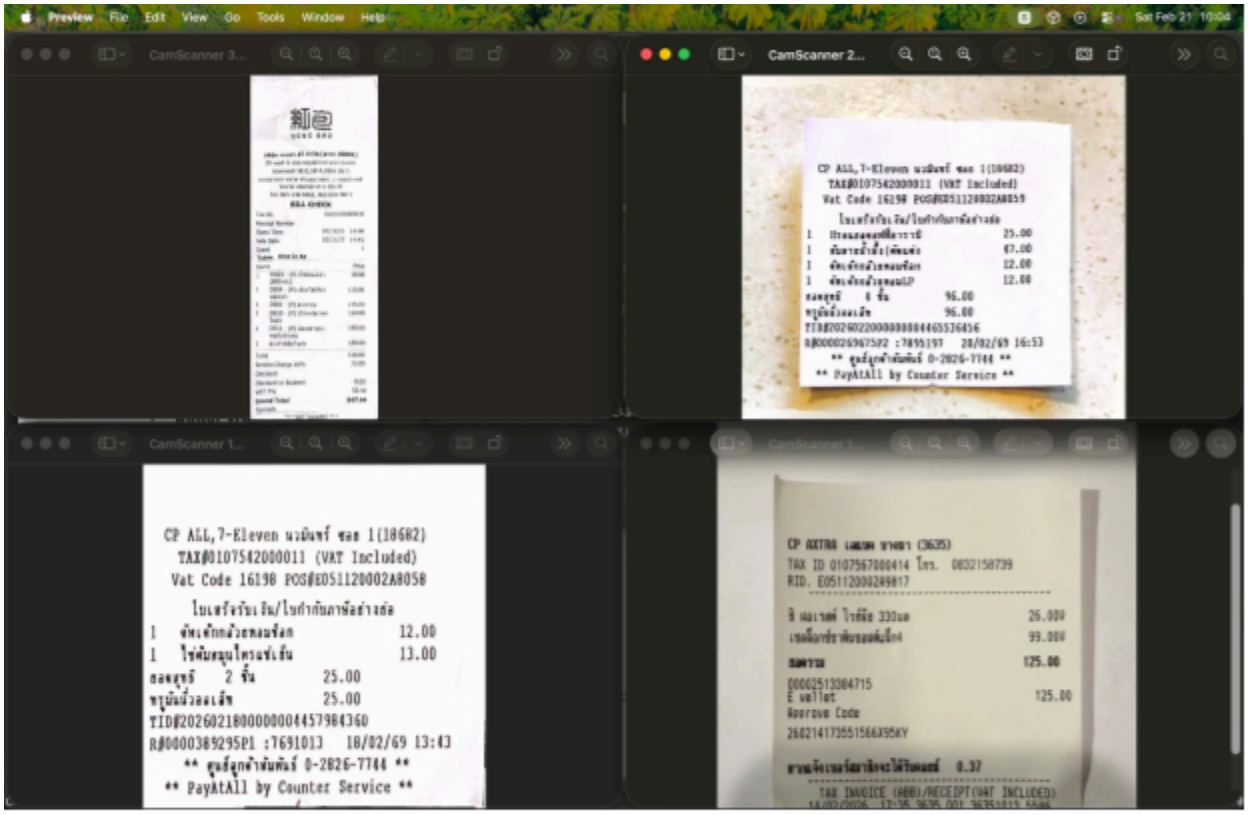


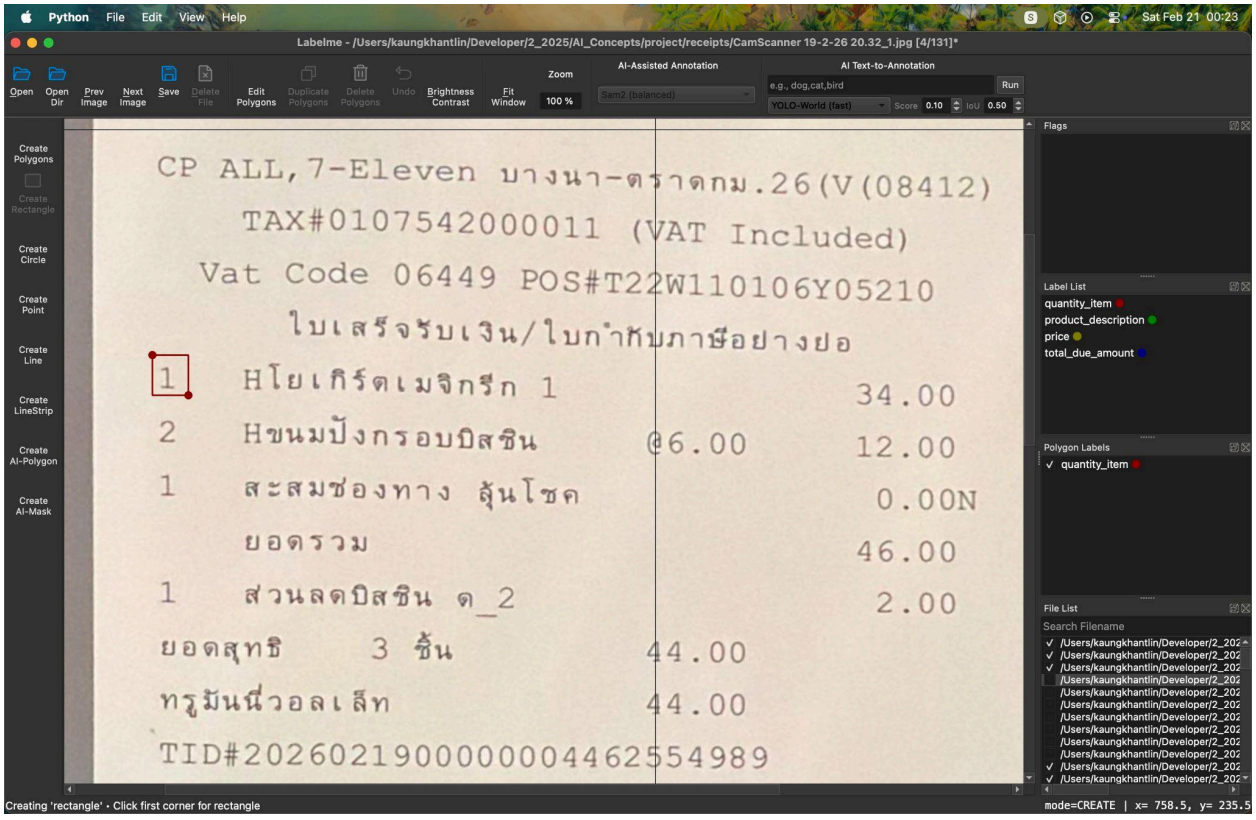
Figure 1: Raw receipts collected from several sources.

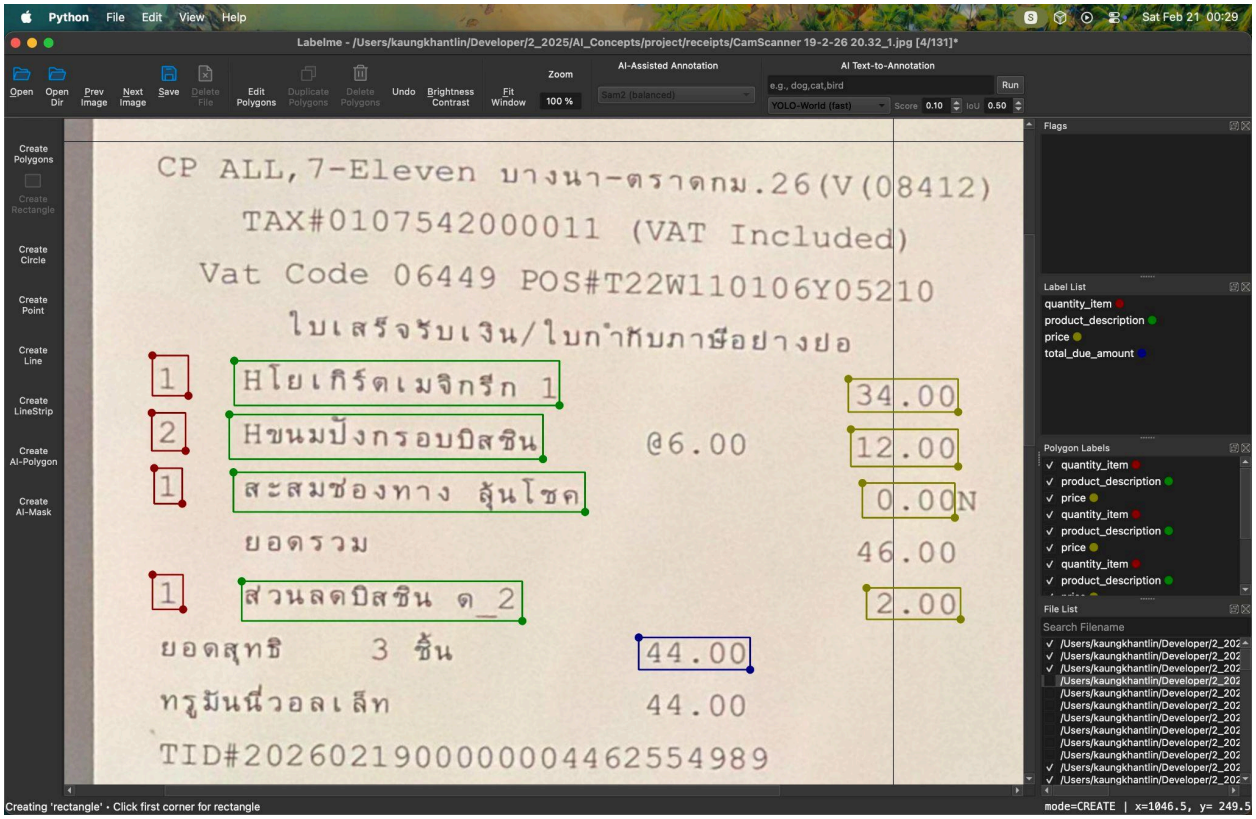
3. Data Annotation (Labeling)

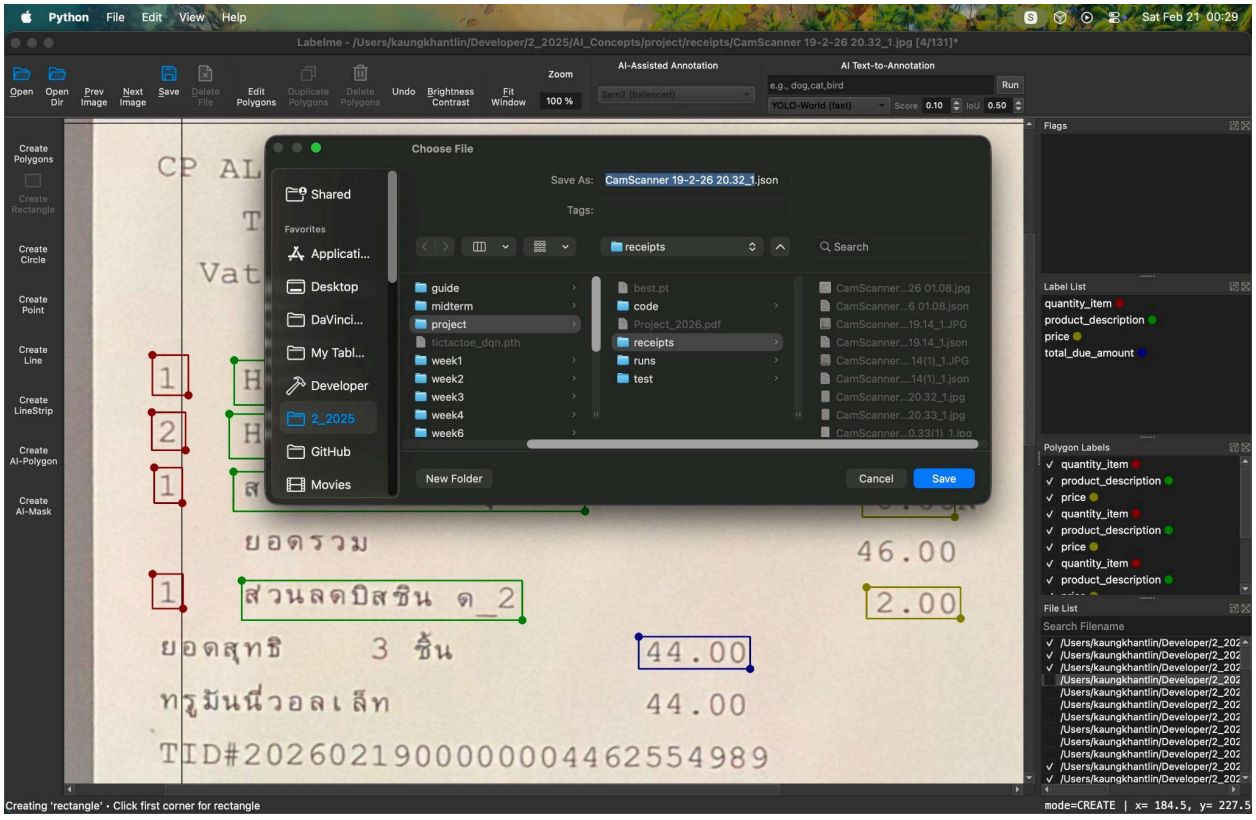
I manually labeled all collected images using the `Labelme` annotation tool. I saved the resulting `.json` files in a local folder named receipts alongside the original images.

To ensure consistency across the dataset, I established the following strict labeling rules:

- **Excluding Currency Symbols:** I drew bounding boxes strictly around the numerical values for price and `total_due_amount` (excluding the "฿" symbol). This prevents the model from getting confused when symbols are missing.
- **Implicit Quantities:** If a receipt did not explicitly state a quantity (implying 1), I skipped the `quantity_item` label for that row rather than drawing a box over empty space, which would confuse the object detection model.
- **Thai Vowels/Tones:** For the `product_description`, I ensured the bounding boxes were tall enough to capture upper and lower Thai tonal marks (e.g.,◌ั, ◌ี, ◌่, ◌้, ◌๊, ◌๋).


Python File Edit View Help
Sat Feb 21 00:23
Labelme - /Users/kaungkhantlin/Developer/2_2025/AI_Concepts/project/receipts/CamScanner 19-2-26 20.32_1.jpg [4/131]*
Open Open Dir Prev Image Next Image Save Delete File Edit Polygons Duplicate Polygons Delete Polygons Undo Brightness Contrast Fit Window Zoom 100 %
AI-Assisted Annotation
Sam2 (balanced)
AI Text-to-Annotation
e.g., dog,cat,bird
Run
YOLO-World (fast)
Score 0.10 IoU 0.50
Create Polygons
Create Rectangle
Create Circle
Create Point
Create Line
Create LineStrip
Create AI-Polygon
Create AI-Mask
CP ALL,7-Eleven บางนา-ตราดกม.26(V(08412)
TAX#0107542000011 (VAT Included)
Vat Code 06449 POS#T22W110106Y05210
ใบเสร็จรับเงิน/ใบกำกับภาษีอย่างย่อ
1 Hโยเกิร์ตเมจิกรีก 1 34.00
2 Hขนมปังกรอบบิสชิน @6.00 12.00
1 สะสมช่องทาง ลุ้นโชค 0.00N
ยอดรวม 46.00
1 ส่วนลดบิสชิน ด_2 2.00
ยอดสุทธิ 3 ชิ้น 44.00
ทรูมันนี่วอลเล็ท 44.00
TID#20260219000000044625549 89
Flags
Label List
quantity_item
product_description
price
total_due_amount
Polygon Labels
quantity_item
File List
Search Filename
Creating 'rectangle' • Click first corner for rectangle
mode=CREATE | x= 758.5, y= 235.5


Python File Edit View Help
Sat Feb 21 00:29
Labelme - /Users/kaungkhantlin/Developer/2_2025/AI_Concepts/project/receipts/CamScanner 19-2-26 20.32_1.jpg [4/131]*
Open Open Dir Prev Image Next Image Save Delete File Edit Polygons Duplicate Polygons Delete Polygons Undo Brightness Contrast Fit Window Zoom 100 %
AI-Assisted Annotation
Sam2 (balanced)
AI Text-to-Annotation
e.g., dog,cat,bird
Run
YOLO-World (fast)
Score 0.10 IoU 0.50
Create Polygons
Create Rectangle
Create Circle
Create Point
Create Line
Create LineStrip
Create AI-Polygon
Create AI-Mask
CP ALL,7-Eleven บางนา-ตราดกม.26(V(08412)
TAX#0107542000011 (VAT Included)
Vat Code 06449 POS#T22W110106Y05210
ใบเสร็จรับเงิน/ใบกำกับภาษีอย่างย่อ
1 Hโยเกิร์ตเมจิกรีก 1 34.00
2 Hขนมปังกรอบบิสชิน @6.00 12.00
1 สะสมช่องทาง ลุ้นโชค 0.00N
ยอดรวม 46.00
1 ส่วนลดบิสชิน ด_2 2.00
ยอดสุทธิ 3 ชิ้น 44.00
ทรูมันนี่วอลเล็ท 44.00
TID#20260219000000044625549 89
Flags
Label List
quantity_item
product_description
price
total_due_amount
Polygon Labels
quantity_item
product_description
price
quantity_item
product_description
price
quantity_item
product_description
File List
Search Filename
Creating 'rectangle' • Click first corner for rectangle
mode=CREATE | x=1046.5, y= 249.5



Figures 2-4: Manual labeling in Labelme.

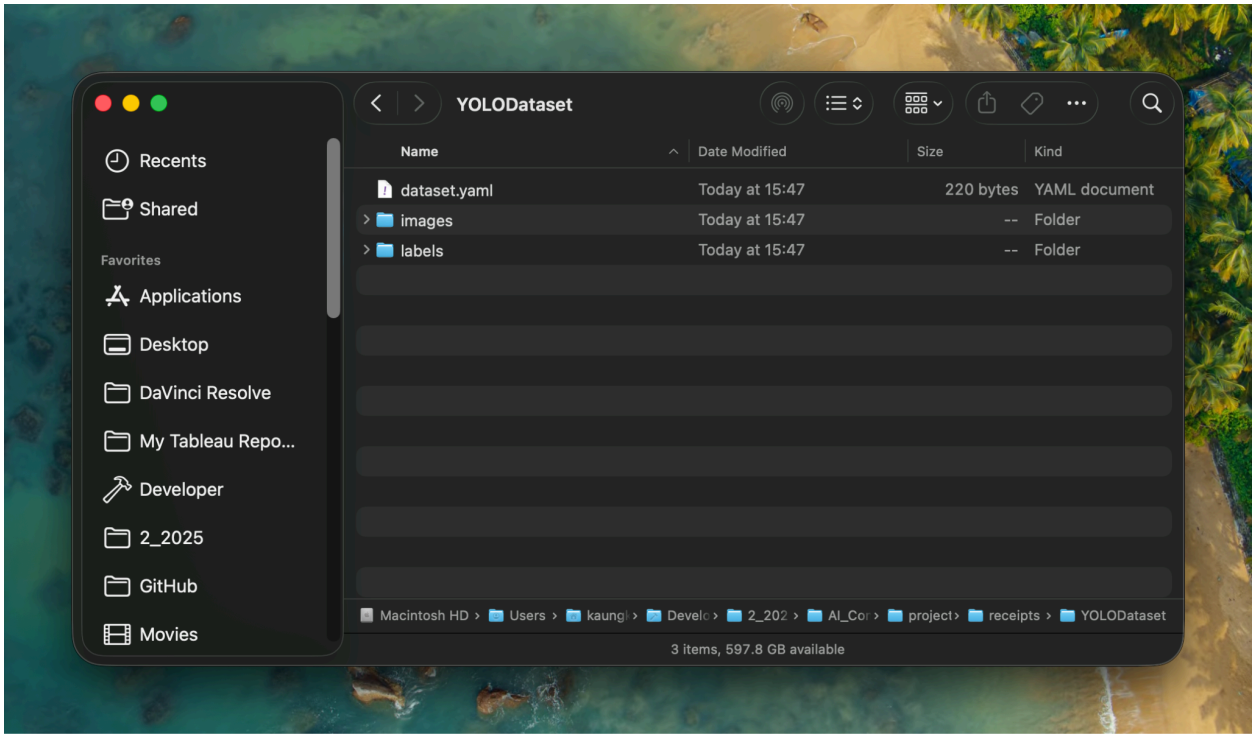
4. Dataset Formatting (Labelme to YOLO)

Since Labelme outputs annotations in JSON polygon format, I needed to convert them into YOLO's required `.txt` bounding box format (`class_id`, `x_center`, `y_center`, `width`, `height`) before training.

I used the `labelme2yolo` Python package to handle the conversion and automatically split my dataset into Training and Validation sets. I opened my terminal and ran the following command:

```
labelme2yolo --json_dir "receipts" --val_size 0.2 --test_size 0.0
--output_format bbox
```

This successfully generated a `YOLODataset` folder with an `80/20 train-validation split`, ensuring I had untouched data to evaluate the model's true performance later.



5. Data Augmentation Strategy

Because my initial dataset consisted of 200 images, I needed to artificially expand the training set to prevent the model from overfitting. I wrote a custom Python script (`augment_yolo.py`) using the `albumentations` library to generate 5 augmented copies of every training image.

Important Design Choice: I intentionally avoided spatial augmentations like HorizontalFlip. Since receipts follow a strict left-to-right reading logic (Description on the left, Price on the right), flipping the image would destroy the spatial context the model needs to distinguish a price from a quantity.

Instead, my augmentation pipeline focuses on simulating bad camera conditions using:

- `SafeRotate` and `Perspective` to simulate slightly crooked scans.
- `RandomBrightnessContrast`, `RGBShift`, and `HueSaturationValue` to simulate different scanner lights, phone camera flashes, and aged paper.
- `ISONoise` to simulate grainy photos from low-quality cameras.

```
transform = A.Compose(
    [
        # 1. GEOMETRY (Conservative)
        # minimal rotation to simulate slightly crooked scans
        A.SafeRotate(limit=3, p=0.5),

        # random perspective simulates taking a photo at an angle
        A.Perspective(scale=(0.02, 0.05), keep_size=True, p=0.3),

        # 2. PIXEL LEVEL (Aggressive is okay here)
        # Simulates different scanner lights or phone camera flashes
        A.RandomBrightnessContrast(brightness_limit=0.2,
contrast_limit=0.2, p=0.5),

        # Simulates different paper ages (yellowing) or printer ink levels
        A.RGBShift(r_shift_limit=20, g_shift_limit=20, b_shift_limit=20,
p=0.3),
        A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=20,
val_shift_limit=20, p=0.3),

        # 3. NOISE (Simulate bad cameras/printing)
        # ISO noise is better than blur for text because edges remain sharp
        A.ISONoise(color_shift=(0.01, 0.05), intensity=(0.1, 0.5), p=0.3),

        # Only use very weak blur if necessary
        A.GaussianBlur(blur_limit=(3, 3), p=0.1),
    ],
    bbox_params=A.BboxParams(format="yolo", min_visibility=0.3,
label_fields=["class_labels"])
)
```

After configuring the paths to point to the output from Step 4, I ran the script, successfully generating a finalized `YOLODataset_aug` folder ready for model training.

6. Cloud Environment Setup (Google Colab)

Training an object detection model especially a larger architecture like `YOLO26l` locally on a `CPU` is highly inefficient. Therefore, I utilized Google Colab to access cloud-based `GPU` acceleration.

First, I compressed my finalized `YOLODataset_aug` folder into a `.zip` file and uploaded it to my Google Drive. Inside my Colab notebook, I mounted my Drive and used the following command to extract the dataset directly into the local `/content/` directory of the Colab instance:

```
!unzip -q "/content/drive/MyDrive/Project/YOLODataset_aug.zip" -d /content/
```

Justification: Extracting the files to the local Colab instance rather than reading them directly from Google Drive during training drastically reduces `I/O` bottlenecks and speeds up the training time per epoch.

I also verified the hardware allocation by running `!nvidia-smi`, which confirmed Google Colab assigned me a `Tesla T4 GPU`.

7. Dataset Configuration Update

Because the file paths on my local macOS environment differ from the Linux environment in Google Colab, the original dataset.yaml file generated by `labelme2yolo` would cause "dataset not found" errors.

To solve this, I wrote a `Python` snippet within the notebook to dynamically read the original `YAML` file and rewrite it as `data_colab.yaml`, automatically correcting the `path`, `train`, and `val` directories to point to the Colab environment:

```
import yaml

src = "/content/YOLODataset_aug/dataset.yaml"
with open(src, "r", encoding="utf-8") as f:
    data = yaml.safe_load(f)

# Force correct path for Colab Linux environment
data["path"] = "/content/YOLODataset_aug"
data["train"] = "images/train"
```

```
data["val"] = "images/val"

out = "/content/data_colab.yaml"
with open(out, "w", encoding="utf-8") as f:
    yaml.safe_dump(data, f, sort_keys=False)
```

8. Model Training Setup

With the dataset structured and the environment ready, I installed the `ultralytics` package to initialize the YOLO training pipeline.

For the model architecture, I selected `yolo26l.pt` (the Large variant). While heavier than the nano or small versions, the large model is better suited for extracting fine-grained visual details, such as the small tonal marks in Thai text or distinguishing between a price and a product code.

I executed the training with the following command and hyperparameters:

```
!yolo detect train \
  model=yolo26l.pt \
  data=/content/data_colab.yaml \
  epochs=200 imgsz=640 batch=32 device=0 \
  project="/content/drive/MyDrive/Project/weight" \
  name="yolo26l_custom" \
  save=True save_period=5 \
  mosaic=1.0 close_mosaic=10 \
  fliplr=0.0 \
  hsv_h=0.03 hsv_s=0.6 hsv_v=0.5 \
  degrees=10 translate=0.1 scale=0.5 \
  mixup=0.1
```

Key Hyperparameter Decisions:

`epochs=200`: Extended the training time to allow the larger model to fully converge on the augmented dataset.

`fliplr=0.0`: Explicitly disabled horizontal flipping. As established during the offline augmentation phase, flipping a receipt destroys the spatial logic required to identify line items correctly.

`mosaic=1.0`: Enabled mosaic augmentation to stitch multiple receipts together during training. This forces the model to learn the contextual relationships between text blocks rather than memorizing their absolute position on the page.

`save_period=5`: Ensures the model saves checkpoint weights every 5 epochs directly to my Google Drive, preventing data loss if the Colab session disconnects.

9. Model Training Execution

Once the training was initialized, the YOLO framework processed the augmented dataset using Google Colab's `Tesla T4 GPU`.

While I set the maximum epochs to 200, the Ultralytics framework utilizes a built-in Early Stopping mechanism to prevent overfitting. The training automatically halted after `168 epochs` (taking approximately `2.254 hours`). The framework detected that the model's performance had peaked earlier and correctly identified that the weights from `Epoch 68` provided the highest validation accuracy.

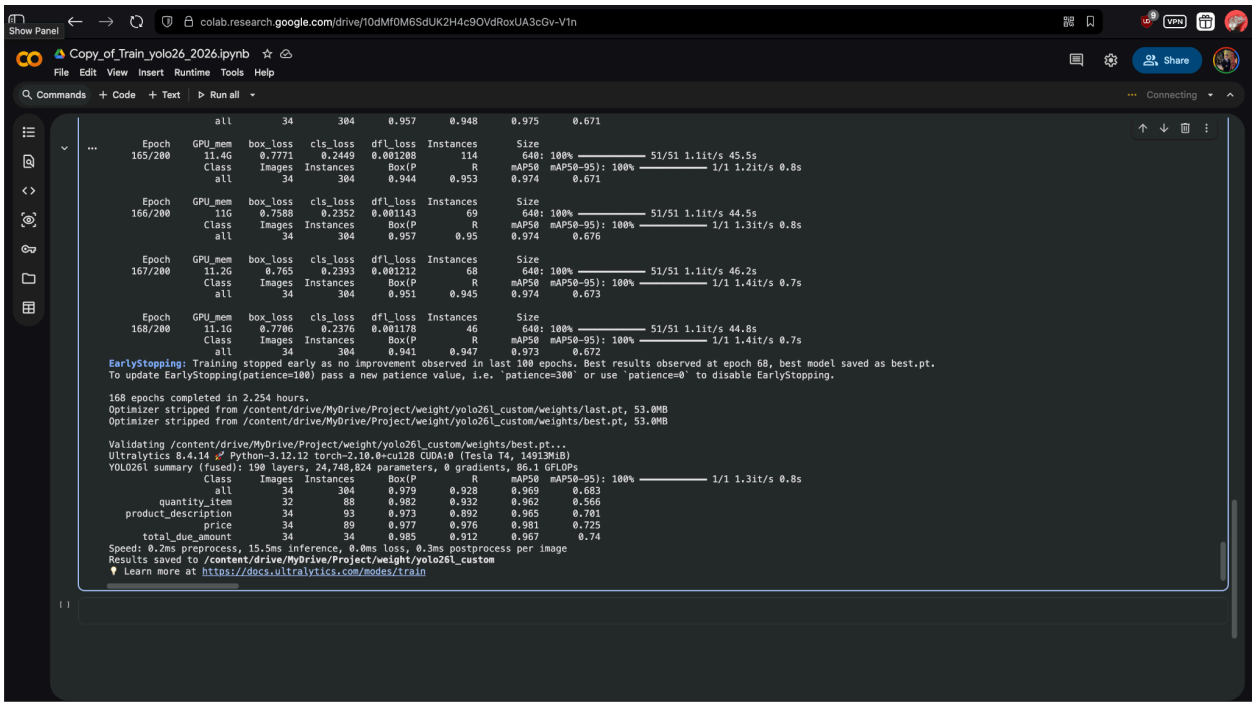


Figure 5: Finished training model in Google Colab snapshot.

10. Model Evaluation & Results

The model's performance on the validation set was outstanding, proving that the custom Albumentations pipeline successfully expanded the dataset without confusing the model's spatial awareness.

The final evaluation yielded an impressive overall `mAP50` of 96.9% (0.969).

Breaking down the precision by specific classes:

`price`: 98.1% `mAP50`

`total_due_amount`: 96.7% `mAP50`

`product_description`: 96.5% `mAP50`

`quantity_item`: 96.2% `mAP50`

Analysis: The price class achieved the highest accuracy, likely because prices on Thai receipts (especially 7-Eleven) are consistently formatted on the far-right edge of the paper. Despite the visual similarity between an individual price and the `total_due_amount`, the model successfully learned to differentiate them based on their vertical positioning (context), achieving over 96% accuracy for both.

Furthermore, the `yolo26l` (Large) architecture successfully detected the `quantity_item` targets (96.2%), which are often difficult for smaller models to catch since they are usually just a single, isolated digit.

11. Confusion Matrix & Error Analysis

To visually analyze where the model succeeds and where it makes false predictions, I evaluated both the absolute and normalized confusion matrices (confusion_matrix.png and confusion_matrix_normalized.png), alongside the training results graph generated during the validation phase.

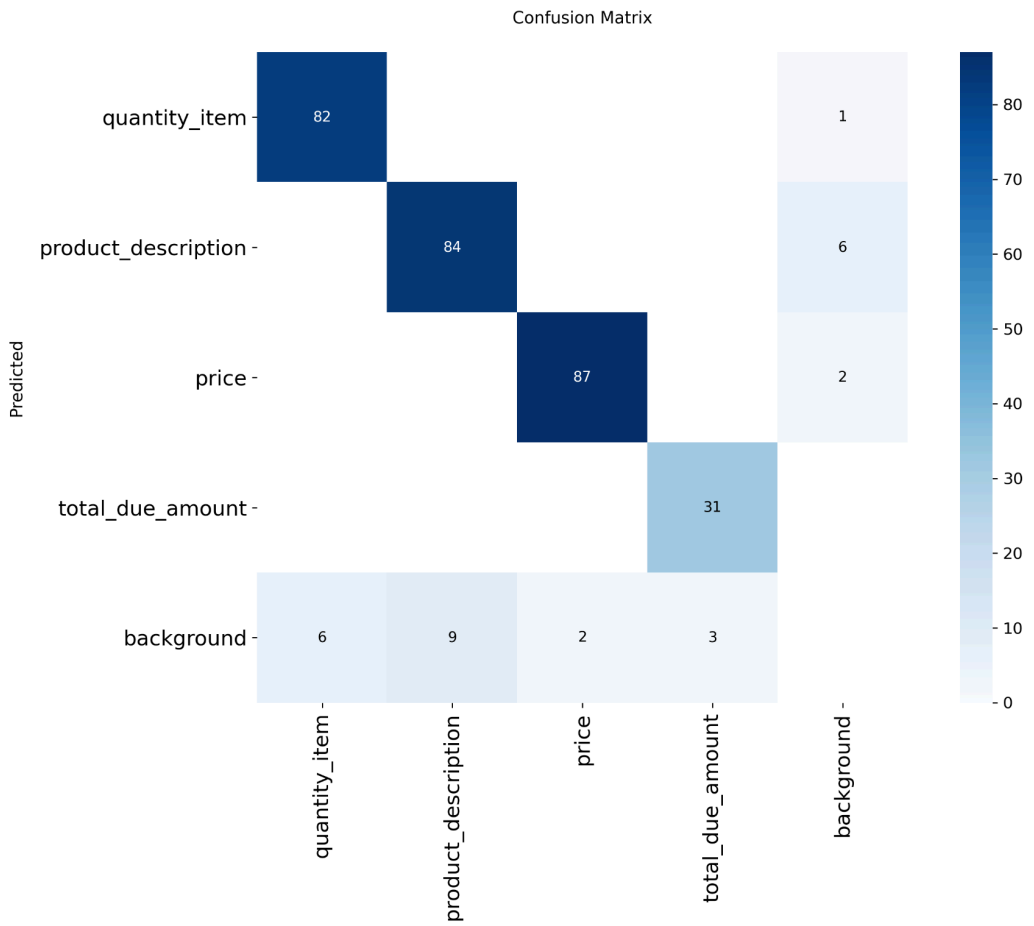


Figure 6: Absolute Confusion Matrix showing the exact count of predictions.

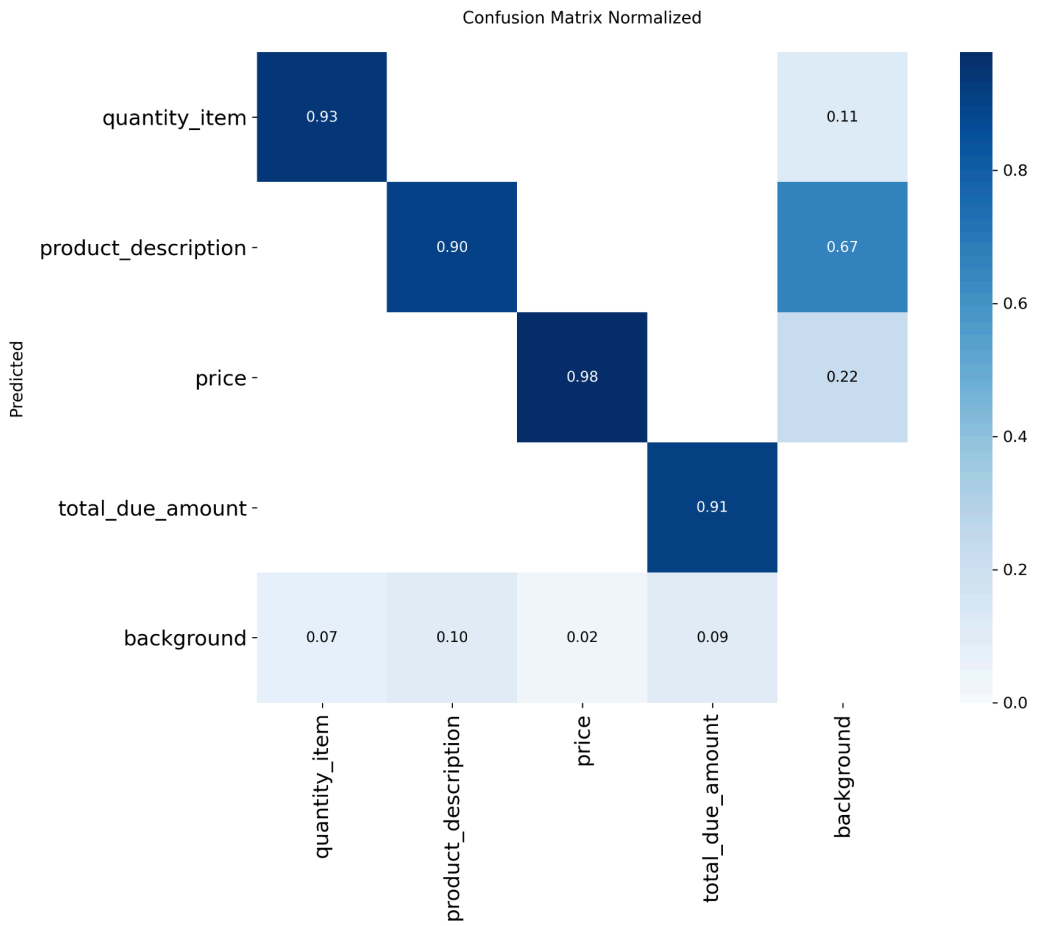


Figure 7: Normalized Confusion Matrix showing the percentage distribution of predictions.

Both matrices show a strong dark diagonal, signifying high True Positives across all four classes. Including both figures is key: the absolute matrix reveals an exceptionally low raw number of background false positives (predicting a class where there is only white space). This minimal background error rate demonstrates that the model's bounding boxes are very tight and accurate, which is crucial for clean handoff to an OCR engine.

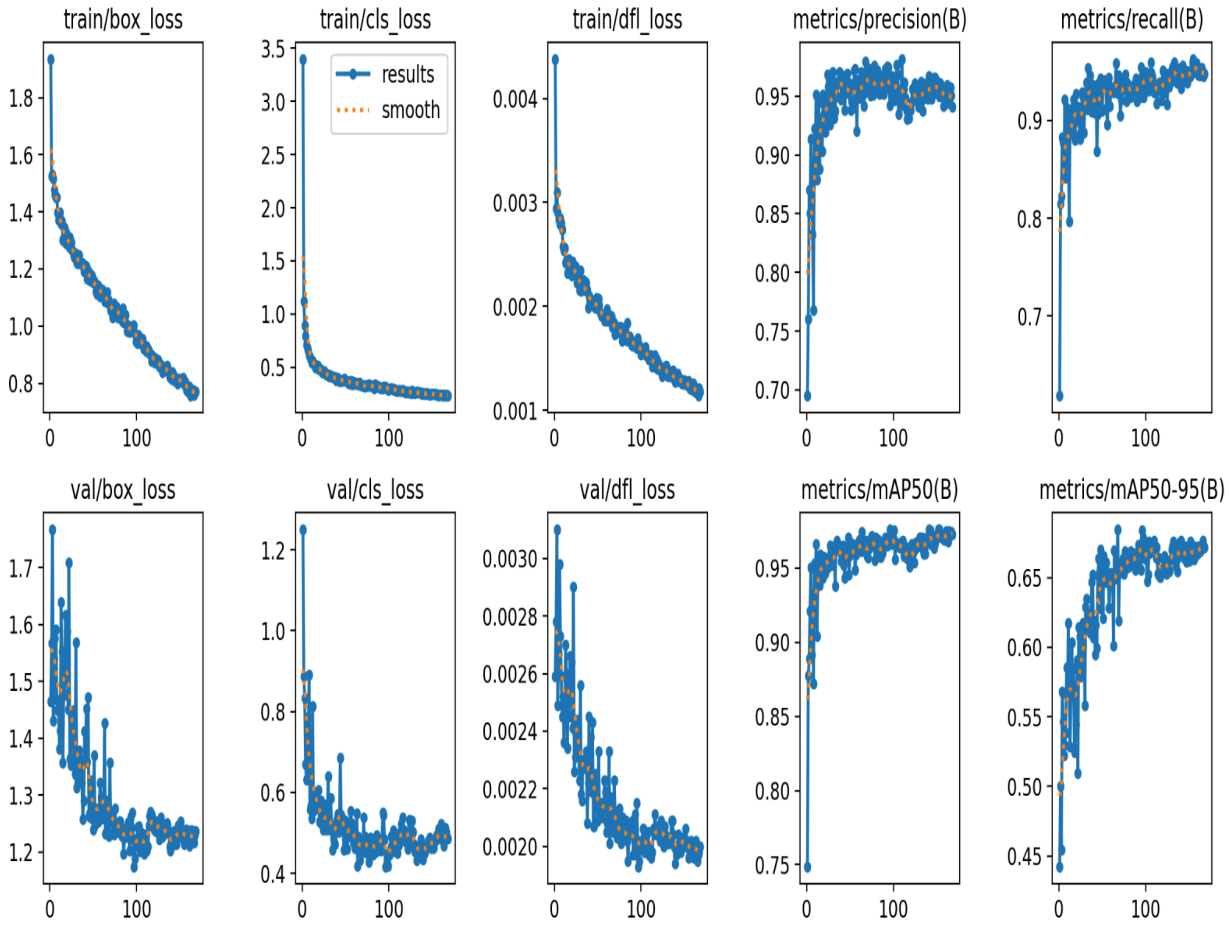


Figure 8: Training results showing a smooth, continuous decrease in bounding box and classification loss.

12. Final Output & Conclusion

Because the model triggered Early Stopping, the Ultralytics framework automatically stripped the optimizer states to reduce the file size (to 53.0MB) and saved the highest-performing checkpoint.

This final, ready-to-use model file `best.pt` was automatically exported to my Google Drive directory. By systematically collecting real-world data, manually establishing strict labeling rules in Labelme, applying geometrically-safe augmentations, and leveraging cloud-based transfer learning, I successfully built a highly accurate object detection model tailored for Thai tax invoices. This exact model file, alongside this detailed workflow report, serves as the final submission for this project.